

Application Responses - Senior Backend Engineer (Elixir) - Remote

This document contains tailored responses for the written questions in the Remote job application.

1. What makes you interested in working with Remote?

I am deeply drawn to Remote for two primary reasons: its mission of democratizing global talent and its pioneering, forward-thinking approach to engineering delivery.

First, as someone who has worked in distributed, multicultural teams (such as Citibank and Perficient/Caterpillar), I have experienced firsthand the incredible value of global collaboration. Remote is solving one of modern business's most significant challenges—removing geographical barriers so that companies can recruit and manage teams compliantly, bringing life-changing opportunities and local wealth to talent worldwide.

Second, I am highly inspired by Remote's explicit commitment to innovation and AI. Your mandate to *“redesign how engineering work ships with autonomous agents as the default execution layer”* perfectly mirrors my own software engineering philosophy. Over the past few years, I have actively operated and customized agentic harnesses (such as Pi.dev and opencode) in my daily workflow to automate repetitive coding tasks while applying rigorous human architectural oversight, testing, and systemic verification. Working at a company that is actively building these agentic workflows into the core engineering fabric is exactly where I want to focus my career.

2. Remote is a value-driven company. Have you seen our values page? (<https://remotecom.notion.com/d43475e8895c4b7c811490aef64782b3>) What resonates with you most?

Of Remote's core values, **Ownership** and **Kindness** resonate with me most deeply.

- **Ownership:** In a fully asynchronous, globally distributed environment, ownership is the cornerstone of success. To me, ownership means defaulting to action, writing clear specifications, defining precise execution plans, and ensuring robust verification loops without needing daily standups or constant supervision. It is about being fully accountable for the outcomes—from understanding stakeholder requirements to shipping secure, reliable, and scalable code.
- **Kindness:** Innovation and high performance can only thrive in a supportive, empathetic, and inclusive workspace. Practicing kindness means active listening, providing constructive and encouraging feedback during code reviews, mentoring colleagues, and fostering an environment where team members feel safe to challenge dogmas and propose bold improvements.

The synergy of these values creates an incredible work-life balance, allowing engineers to put their family and life first while delivering world-class outcomes asynchronously.

3. Our backend is built in Elixir. Could you share details about your experience with the language and your thoughts on it? If you don't have experience with Elixir, how would you approach learning it, and what do you find appealing about functional programming?

I have hands-on, professional experience developing and architecting with **Elixir**, **Phoenix**, and **Ash Framework** in my recent role as a Lead Software Engineer for a Mobility Startup.

My experience with Elixir has made it one of my favorite backend languages. I find several of its core paradigms incredibly compelling: * **Concurrency and Fault Tolerance:** The Erlang VM (BEAM), lightweight processes, and OTP supervisor trees make building self-healing, highly concurrent, and fault-tolerant applications incredibly simple compared to traditional thread-based ecosystems. * **Functional Purity & Predictability:** The combination of immutability, pattern matching, and the pipe operator (`|>`) leads to codebases that are clean, highly readable, and exceptionally easy to test and debug. * **Rapid Productivity:** Phoenix and Ash Framework enable

rapid full-stack iteration without sacrificing the performance, low latency, and low memory consumption necessary for high-throughput production environments.

In my mobility startup role, I designed and maintained Elixir-based backends that integrated complex third-party APIs and managed persistent database transactions with PostgreSQL, demonstrating how Elixir can power critical infrastructure smoothly.

4. Can you share an example of a product or project you led through collaboration with stakeholders, product, and design? Tell us about it.

In my role as a **Tech Lead** at **Seven IT SpA**, I led the end-to-end development of a comprehensive Healthcare Management System for hospital administration and clinical staff.

- **The Challenge:** The project required digitizing complex clinical workflows (medicine, nursing, physical therapy), tracking real-time patient care, managing payments, and producing administrative reports.
- **Collaboration & Leadership:** I collaborated closely with non-technical stakeholders (hospital administrators, medical chiefs) and product owners to translate clinical regulations and operational requirements into precise technical specifications. I also worked hand-in-hand with frontend designers to create an intuitive, accessible user experience.
- **Execution:** I led and mentored a team of 4 developers. We designed a robust backend REST API and integrated it with a modern frontend (using JavaScript and Vuetify) and a PostgreSQL database.
- **The Outcome:** We successfully shipped the platform on time, which successfully digitized healthcare management across multiple departments, reduced payment processing errors, and automated the generation of certified PDF medical reports.

5. Tell us about your proudest achievement from your last two years of work. Why are you proud of it?

My proudest achievement from my recent work was **successfully redesigning our software delivery workflow to incorporate autonomous software agents as our default execution layer**, resulting in a **300% increase in development velocity** with zero regressions.

Instead of treating AI simply as a chat assistant, I designed and operationalized custom agentic workflows using **Pi.dev** and **opencode** harnesses: 1. **The Workflow:** I established a rigorous end-to-end loop: **spec** → **plan** → **execute** → **verify**. 2. **Systemic Verification:** I built automated verification loops where agents were directed to write code, generate tests, and run static analyses, while I acted as the human-in-the-loop architect—providing high-level constraints, conducting deep code reviews, and applying guardrails. 3. **Primitives & Scalability:** I developed reusable agentic primitives in our codebase, allowing the entire engineering team to safely delegate repetitive tasks across multiple business domains.

I am incredibly proud of this because it proved that when combined with human architectural judgment and rigorous verification loops, autonomous software agents can dramatically accelerate product delivery while maintaining absolute code quality, security, and scalability.